COMPETITIVE PROGRAMMING

De Zéro à Champion Mondial

Le guide ultime, complet et profond

Algorithmes · Structures de données · Stratégie · Mindset

"La différence entre un expert et un débutant,
c'est des milliers d'heures de problèmes résolus."

Guide structuré en 10 Parties • 40+ Chapitres • 200+ Techniques

TABLE DES MATIÈRES

PARTIE 0 — FONDATIONS & MINDSET

Ch.1 · Le mindset du champion

- Discipline
- Gestion du temps
- Resilience

Ch.2 · Environnement de travail

- Choix du langage
- IDE & outils
- Template de base

Ch.3 · Complexité algorithmique

- Big-O
- Analyse amortie
- Règles pratiques

PARTIE 1 — MATHÉMATIQUES POUR CP

Ch.4 · Arithmétique & Théorie des nombres

- GCD/LCM
- Crible
- Modular arithmetic

Ch.5 · Combinatoire & Probabilités

- Binôme
- Principe d'inclusion-exclusion
- Espérance

Ch.6 · Algèbre linéaire & Matrices

- Exponentiation de matrices
- Systèmes linéaires

Ch.7 · Théorie des jeux

- Nim
- Sprague-Grundy
- Jeux combinatoires

PARTIE 2 — STRUCTURES DE DONNÉES

Ch.8 · Structures de base

- Stack
- Queue
- Deque
- Priority Queue

Ch.9 · Segment Tree & BIT

- Point update
- Range query
- Lazy propagation

Ch.10 · DSU / Union-Find

- Path compression
- Union by rank
- Applications

Ch.11 · Structures avancées

- Sparse Table
- Treap
- Trie
- Hashing

PARTIE 3 — TECHNIQUES FONDAMENTALES

Ch.12 · Two Pointers & Sliding Window

- Fenêtre fixe
- Fenêtre variable
- Multi-pointeurs

Ch.13 · Binary Search

- Sur la réponse
- Sur une fonction monotone
- Ternary search

Ch.14 · Greedy

- Preuve d'optimalité
- Exchange argument
- Patterns classiques

Ch.15 · Divide & Conquer

- Merge sort
- CDQ
- Optimisation DP

PARTIE 4 — DYNAMIC PROGRAMMING

Ch.16 · Introduction & cadre général

- États
- Transitions
- Base cases

Ch.17 · DP classiques

- Knapsack
- LIS
- LCS
- Edit distance

Ch.18 · DP sur intervalles & arbres

- Matrix chain
- Tree DP

Ch.19 · DP avec bitmask

- TSP
- Assignment problem

Ch.20 · Optimisations DP

- CHT
- D&C; DP
- Knuth
- Monotone queue

PARTIE 5 — GRAPHES

Ch.21 · Représentation & Parcours

- Adj list
- BFS
- DFS

- Applications

Ch.22 · Plus courts chemins

- Dijkstra
- Bellman-Ford
- SPFA
- Floyd-Warshall

Ch.23 · Arbres couvrants

- Kruskal
- Prim
- Propriétés

Ch.24 · Graphes avancés

- SCC
- Bridges
- Articulation points
- 2-SAT

Ch.25 · Flots & Matching

- Max-flow
- Min-cut
- Bipartite matching
- Hungarian

PARTIE 6 — ALGORITHMES SUR LES CHAÎNES

Ch.26 · Hashing de chaînes

- Polynomial hashing
- Double hashing

Ch.27 · KMP & Z-function

- Pattern matching
- Applications

Ch.28 · Suffix Array & Suffix Automaton

- Construction
- LCP
- Applications

Ch.29 · Aho-Corasick

- Automate
- Multi-pattern matching

PARTIE 7 — GÉOMÉTRIE COMPUTATIONNELLE

Ch.30 · Primitives géométriques

- Points
- Vecteurs
- Produit vectoriel

Ch.31 · Convex Hull & balayage

- Graham scan
- Andrew monotone chain

Ch.32 · Intersection & distances

- Segment intersection
- Plus proche paire

PARTIE 8 — TECHNIQUES AVANCÉES

Ch.33 · Small-to-large / DSU on tree

- Algorithme d'Euler tour

Ch.34 · Mo's algorithm

- Offline queries
- Mo avec mises à jour

Ch.35 · Randomisation & Hashing avancé

- Bloom filter
- Random tricks

Ch.36 · Théorie des nombres avancée

- CRT
- BSGS
- Primitive root

PARTIE 9 — STRATÉGIE & COMPÉTITION

Ch.37 · Lire et analyser un problème

- Identifier le type
- Cas limites

Ch.38 · Plan d'entraînement annuel

- Roadmap par niveau
- Ressources

Ch.39 · Codeforces, ICPC, IOI

- Stratégie de contest
- Gestion du stress

Ch.40 · Vers le niveau Grandmaster

- Étude systématique
- Mindset élite

PARTIE 0 — FONDATIONS & MINDSET

"Avant d'écrire une seule ligne de code, le champion se construit dans sa tête."

Chapitre 1 — Le Mindset du Champion

Le competitive programming est autant un sport mental qu'un exercice technique. Les meilleurs joueurs mondiaux — Petr Mitrichev, tourist (Gennady Korotkevich), Um_nik — partagent tous des traits communs qui transcendent leurs connaissances algorithmiques.

1.1 — Les piliers du mindset

- **Discipline quotidienne** : résoudre des problèmes CHAQUE JOUR, même 1 seul, vaut mieux que 10 problèmes le weekend.
- **Accepter l'échec comme outil** : un WA (Wrong Answer) ou TLE (Time Limit Exceeded) n'est pas un échec — c'est une donnée.
- **Penser avant de coder** : les champions passent 60-70% du temps à réfléchir sur papier, pas à taper.
- **Lire les editoriaux** : après 45 min sans avancer, lire la solution et comprendre POURQUOI tu n'y as pas pensé.
- **Progression mesurable** : suivre son rating Codeforces, son upsolving, ses AC/WA ratio.

1.2 — Gestion du temps en contest

En compétition, chaque minute compte. Voici la stratégie des top players :

- Lire TOUS les problèmes en 10 min → identifier les faciles.
- Commencer par le plus facile, même trivial.
- Règle des 20/40 min : si bloqué après 20 min → passer. Si bloqué après 40 min → passer définitivement en contest.
- Toujours soumettre même si non certain (en Div. 2 A/B surtout).
- Ne jamais débugger pendant plus de 15 min → réécrire proprement.

CONSEIL : Crée un chronomètre mental : si tu n'as pas d'idée après 20 min, change de problème.

1.3 — Résilience et progression à long terme

La courbe de progression en CP est logarithmique : les premiers 6 mois sont difficiles, puis ça s'accélère. Les plateaux sont normaux — ils signifient que tu absorbes des concepts complexes.

- Div. 2 D débloqué généralement après 6-12 mois d'entraînement sérieux.
- Niveau Expert (1600-1900) : 1-2 ans de pratique régulière.
- Niveau Candidat Master+ (1900+) : 2-4 ans + étude profonde.
- Grandmaster (2400+) : top 1% mondial, années de dédicace.

Chapitre 2 — Environnement de Travail

2.1 — Choix du langage

Langage	Vitesse	STL/Bibliothèques	Recommandé pour
C++17/20	■ Maximal	■ STL complète	Compétition (1er choix)
Python 3	■ Lent (10-50x)	■ Riche	Apprentissage, prototypage
Java	■ Rapide	■ Bonne	ICPC (si habitué)
Kotlin	■ Rapide	■ Bonne	Alternative Java

■■ ATTENTION : Python dépasse souvent le time limit en CP. C++17 est le standard mondial.

2.2 — Template C++ de base (indispensable)

Tout compétiteur sérieux a un template personnel qu'il maîtrise parfaitement :

```
#include <bits/stdc++.h> using namespace std; typedef long long ll; typedef pair<int,int> pii; typedef vector<int> vi; #define all(x) x.begin(), x.end() #define pb push_back #define fi first #define se second const int MOD = 1e9 + 7; const int INF = 1e9; const ll LINF = 1e18; void solve() { // ton code ici } int main() { ios_base::sync_with_stdio(false); cin.tie(NULL); int t; cin >> t; while (t--) solve(); return 0; }
```

■ CONSEIL : ios_base::sync_with_stdio(false) + cin.tie(NULL) est OBLIGATOIRE pour éviter le TLE sur les I/O lents.

2.3 — Outils essentiels

- ■ **IDE** : VS Code + extensions C++, ou CLion (payant mais puissant).
- ■ **Stress testing** : écrire un générateur de cas + solution brute pour détecter les bugs.
- ■ **Codeforces Polygon** : pour créer tes propres tests.
- ■ **CP-algorithms.com** : référence algorithmique de confiance.
- ■ **CSES Problem Set** : 300 problèmes classiques organisés par catégorie.
- ■ **AtCoder** : excellents problèmes de qualité, très bien catégorisés.

Chapitre 3 — Complexité Algorithmique

Comprendre la complexité est LA compétence la plus importante. Elle te permet de savoir IMMÉDIATEMENT si ton algorithme passera le time limit avant même de le coder.

3.1 — Règle d'or des contraintes

Contrainte n	Complexité max acceptable	Algorithme typique
n ≤ 10	O(n!) ou O(2^n)	Permutations, brute force

n ≤ 20	O(2^n · n)	Bitmask DP
n ≤ 500	O(n^3)	Floyd-Warshall, DP O(n^3)
n ≤ 5 000	O(n^2)	Brute force quadratique
n ≤ 10^5	O(n log n)	Tri, Segment Tree, BIT
n ≤ 10^6	O(n) ou O(n log n)	Two pointers, BFS/DFS
n ≤ 10^9	O(log n) ou O(√n)	Binary search, factorisation

3.2 — Calcul rapide du nombre d'opérations

Un ordinateur moderne effectue environ 10^8 à 10^9 opérations simples par seconde. Avec un time limit de 2 secondes :

- ■ Budget : ~2 × 10^8 opérations
- ■ n = 10^5, O(n log n) = ~1.7 × 10^6 → ■ Très OK
- ■ n = 10^5, O(n^2) = 10^10 → ■ TLE assuré
- ■ n = 5000, O(n^2) = 25 × 10^6 → ■ OK

■ CONSEIL : Avant de coder, calcule TOUJOURS le nombre d'opérations de ton algorithme.

3.3 — Complexité amortie

Certains algorithmes ont une complexité 'par opération' apparemment élevée mais amortie sur n opérations, elle est O(1) ou O(log n). Exemple : push_back sur un vector C++ est O(1) amorti même si parfois O(n).

PARTIE 1 — MATHÉMATIQUES POUR CP

"Les mathématiques sont le langage secret des algorithmes."

Chapitre 4 — Arithmétique & Théorie des Nombres

4.1 — PGCD, PPCM et algorithme d'Euclide

Le PGCD (Plus Grand Commun Diviseur) est fondamental. L'algorithme d'Euclide le calcule en O(log(min(a,b))) :

```
// PGCD récursif ll gcd(ll a, ll b) { return b == 0 ? a : gcd(b, a % b); } // PGCD itératif (plus rapide en pratique) ll gcd(ll a, ll b) { while(b) { a %= b; swap(a,b); } return a; } // PPCM (attention au overflow !) ll lcm(ll a, ll b) { return a / gcd(a, b) * b; } // En C++17 : __gcd(a, b) ou std::gcd(a, b) #include<numeric>
```

- **Propriétés** : gcd(a,b) = gcd(b, a%b), gcd(a,0) = a
- **Application** : vérifier si des fractions sont irréductibles, synchronisation de cycles.
- **Euclide étendu** : trouve x, y tels que ax + by = gcd(a,b)

4.2 — Arithmétique modulaire

Incontournable quand les résultats deviennent énormes. La plupart des problèmes demandent réponse mod 10^9+7 (premier).

```
const ll MOD = 1e9 + 7; // Addition et multiplication sûres ll add(ll a, ll b) { return (a + b) % MOD; } ll mul(ll a, ll b) { return (a % MOD * b % MOD) % MOD; } // Exponentiation rapide : a^b mod m en O(log b) ll power(ll base, ll exp, ll mod) { ll result = 1; base %= mod; while (exp > 0) { if (exp & 1) result = result * base % mod; base = base * base % mod; exp >>= 1; } return result; } // Inverse modulaire (si mod est premier) ll inv(ll a, ll mod) { return power(a, mod - 2, mod); } // Division modulaire ll divide(ll a, ll b) { return mul(a, inv(b, MOD)); }
```

■ CONSEIL : 10^9+7 est premier → on peut utiliser le petit théorème de Fermat pour l'inverse.

4.3 — Crible d'Ératosthène

Trouver tous les nombres premiers jusqu'à N en O(N log log N) :

```
vector<bool> is_prime(N+1, true); is_prime[0] = is_prime[1] = false; for (int i = 2; i * i <= N; i++) { if (is_prime[i]) { for (int j = i*i; j <= N; j += i) is_prime[j] = false; } } // Variante : crible linéaire pour factorisation vector<int> smallest_prime(N+1, 0); vector<int> primes; for (int i = 2; i <= N; i++) { if (!smallest_prime[i]) { smallest_prime[i] = i; primes.push_back(i); } for (int p : primes) { if (p > smallest_prime[i] || i*p > N) break; smallest_prime[i*p] = p; } }
```

4.4 — Factorisation et diviseurs

```
// Factorisation en O(√n) map<int,int> factorize(int n) { map<int,int> f; for (int i = 2; i * i <= n; i++) while (n % i == 0) { f[i]++; n /= i; } if (n > 1) f[n]++; return f; } // Nombre de diviseurs depuis factorisation // n = p1^a1 * p2^a2 * ... => nb_div = (a1+1)(a2+1)... // Tous les diviseurs de n en O(√n) vector<int> divisors(int n) { vector<int> d; for (int i = 1; i*i <= n; i++) { if (n%i==0) { d.pb(i); if(i!=n/i)
```

```
d.pb(n/i); } } sort(all(d)); return d; }
```

Chapitre 5 — Combinatoire & Probabilités

5.1 — Coefficients binomiaux

C(n,k) = n! / (k! × (n-k)!). Précalcul en O(n^2) avec le triangle de Pascal ou O(n) avec les factorielles et inverses modulaires :

```
const int MAXN = 2e5 + 5; ll fact[MAXN], inv_fact[MAXN]; void precompute() { fact[0] = 1; for (int i = 1; i < MAXN; i++) fact[i] = fact[i-1] * i % MOD; inv_fact[MAXN-1] = power(fact[MAXN-1], MOD-2, MOD); for (int i = MAXN-2; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % MOD; } ll C(int n, int k) { if (k < 0 || k > n) return 0; return fact[n] % MOD * inv_fact[k] % MOD * inv_fact[n-k] % MOD; }
```

5.2 — Principe d'inclusion-exclusion (PIE)

Pour compter les éléments satisfaisant AU MOINS UNE condition sur k propriétés :

|A■ ∪ A■ ∪ ... ∪ A■| = Σ|A■| - Σ|A■∩A■| + Σ|A■∩A■∩A■| - ...

- Compter les entiers dans [1,n] non divisibles par aucun des premiers p1,...,pk.
- Compter les permutations sans point fixe (dérangements).
- Nombre de fonctions surjectives de A vers B.

5.3 — Nombres de Catalan, Fibonacci et autres séquences

- **Catalan** C■ = C(2n,n)/(n+1) : arbres binaires, parenthésages valides, chemins.
- **Fibonacci** : F(n) = F(n-1) + F(n-2). Calcul en O(log n) via exponentiation de matrices.
- **Nombres de Stirling** : partitions d'ensembles.
- **Principe des tiroirs** (pigeonhole) : n+1 objets dans n cases → au moins une case ≥ 2.

Chapitre 6 — Algèbre Linéaire & Matrices

6.1 — Exponentiation de matrices

Calculer M^n en O(k^3 log n) où k est la taille de la matrice. Permet de calculer F(n) en O(log n) :

```
typedef vector<vector<ll>> Matrix; Matrix multiply(const Matrix& A, const Matrix& B, ll mod) { int n = A.size(); Matrix C(n, vector<ll>(n, 0)); for (int i = 0; i < n; i++) for (int k = 0; k < n; k++) for (int j = 0; j < n; j++) C[i][j] = (C[i][j] + A[i][k]*B[k][j]) % mod; return C; } Matrix matpow(Matrix M, ll p, ll mod) { int n = M.size(); Matrix result(n, vector<ll>(n, 0)); for (int i = 0; i < n; i++) result[i][i] = 1; // identite while (p > 0) { if (p & 1) result = multiply(result, M, mod); M = multiply(M, M, mod); p >>= 1; } return result; }
```

Chapitre 7 — Théorie des Jeux

7.1 — Jeu de Nim

Nim : n tas de jetons, deux joueurs alternent, retire au moins 1 jeton d'un tas, dernier à jouer gagne.

Théorème de Sprague-Grundy : La position est perdante (P-position) si et seulement si XOR de tous les tas = 0.

```
// Nim simple bool nim_wins(vector<int>& piles) { int xor_sum = 0; for (int p : piles) xor_sum ^= p; return xor_sum != 0; // true = position gagnante pour le joueur courant }
```

7.2 — Valeurs de Grundy (mex)

Pour un jeu général, chaque position a une valeur de Grundy G(pos) = mex({G(successeurs)}).

mex = Minimum EXcludant = le plus petit entier non négatif absent de l'ensemble.

Combinaison de jeux : G(jeu1 + jeu2) = G(jeu1) XOR G(jeu2). Position perdante ■ G = 0.

PARTIE 2 — STRUCTURES DE DONNÉES

Chapitre 8 — Structures de Base

8.1 — Stack, Queue, Deque

- **Stack** (LIFO) : matching de parenthèses, DFS itératif, calcul d'expressions.
- **Queue** (FIFO) : BFS, fenêtre glissante.
- **Deque monotone** : maximum/minimum dans une fenêtre glissante en O(1) amorti.
- **Priority Queue** (Heap) : Dijkstra, tri par sélection optimal, problèmes de scheduling.

```
// Deque monotone : max dans fenêtre de taille k vector<int> sliding_max(vector<int>& a, int k) { deque<int> dq; // stocke les indices vector<int> result; for (int i = 0; i < a.size(); i++) { while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); dq.push_back(i); if (dq.front() <= i - k) dq.pop_front(); if (i >= k - 1) result.push_back(a[dq.front()]); } return result; }
```

Chapitre 9 — Segment Tree & Binary Indexed Tree

9.1 — Segment Tree (arbre de segments)

Structure permettant des requêtes et mises à jour sur des intervalles en O(log n). La structure de données la plus polyvalente en CP.

```
struct SegTree { int n; vector<ll> tree; SegTree(int n) : n(n), tree(4*n, 0) {} void update(int node, int l, int r, int pos, ll val) { if (l == r) { tree[node] = val; return; } int mid = (l + r) / 2; if (pos <= mid) update(2*node, l, mid, pos, val); else update(2*node+1, mid+1, r, pos, val); tree[node] = tree[2*node] + tree[2*node+1]; // merge = somme } ll query(int node, int l, int r, int ql, int qr) { if (qr < l || r < ql) return 0; if (ql <= l && r <= qr) return tree[node]; int mid = (l + r) / 2; return query(2*node, l, mid, ql, qr) + query(2*node+1, mid+1, r, ql, qr); } void update(int pos, ll val) { update(1, 0, n-1, pos, val); } ll query(int l, int r) { return query(1, 0, n-1, l, r); } };
```

9.2 — Lazy Propagation (Propagation paresseuse)

Permet des mises à jour par intervalles en O(log n). Exemple : ajouter une valeur à tous les éléments de [l,r].

- Stocker le 'lazy tag' : la mise à jour en attente.
- Push down : propager le lazy aux enfants avant toute opération.
- Applications : range add, range set, range max/min.

9.3 — Binary Indexed Tree (BIT / Fenwick Tree)

Plus simple et plus rapide que le Segment Tree pour les sommes préfixes avec mises à jour. O(log n) par opération.

```
struct BIT { int n; vector<ll> bit; BIT(int n) : n(n), bit(n+1, 0) {} void update(int i, ll delta) { for (++i; i <= n; i += i & -i) bit[i] += delta; } ll query(int i) { // somme [0..i] ll s = 0; for (++i; i > 0; i -= i & -i) s += bit[i]; return s; } ll
```

```
query(int l, int r) { // somme [l..r] return query(r) - (l > 0 ? query(l-1) : 0); } };
// i & -i = lowest set bit (astuce binaire fondamentale)
```

Chapitre 10 — DSU / Union-Find

Structure pour gérer des composantes connexes avec fusion rapide. O(α(n)) par opération (quasiment O(1)).

```
struct DSU { vector<int> parent, rank_; int components; DSU(int n) : parent(n),
rank_(n, 0), components(n) { iota(all(parent), 0); // parent[i] = i } int find(int x)
{ if (parent[x] != x) parent[x] = find(parent[x]); // path compression return
parent[x]; } bool unite(int x, int y) { x = find(x); y = find(y); if (x == y) return
false; if (rank_[x] < rank_[y]) swap(x, y); parent[y] = x; if (rank_[x] == rank_[y])
rank_[x]++; components--; return true; } bool same(int x, int y) { return find(x) ==
find(y); } };
```

- **Applications** : Kruskal MST, détection de cycles, composantes connexes dynamiques.
- **DSU avec rollback** : pour annuler des unions (offline LCT).
- **DSU on tree** (small-to-large) : optimisation de certains DP sur arbres.

Chapitre 11 — Structures Avancées

11.1 — Sparse Table (RMQ en O(1))

Pour des requêtes de minimum/maximum sur un tableau statique en O(1) après un prétraitement O(n log n).

```
struct SparseTable { vector<vector<int>> sp; vector<int> lg; SparseTable(vector<int>&
a) { int n = a.size(); int LOG = __lg(n) + 1; sp.assign(LOG, vector<int>(n));
lg.resize(n+1); sp[0] = a; for (int j = 1; j < LOG; j++) for (int i = 0; i+(1<<j) <=
n; i++) sp[j][i] = min(sp[j-1][i], sp[j-1][i+(1<<(j-1))]); lg[1] = 0; for (int i = 2;
i <= n; i++) lg[i] = lg[i/2] + 1; } int query(int l, int r) { // min dans [l..r] en
O(1) int j = lg[r-l+1]; return min(sp[j][l], sp[j][r-(1<<j)+1]); } };
```

11.2 — Trie (Arbre de préfixes)

Structure pour stocker et chercher des chaînes efficacement. Très utile pour XOR maximum, auto-complétion.

```
struct Trie { struct Node { int ch[26]; bool end; Node() { fill(ch,ch+26,-1);
end=false; } }; vector<Node> nodes; Trie() { nodes.emplace_back(); } void
insert(string& s) { int cur = 0; for (char c : s) { int x = c - 'a'; if
(nodes[cur].ch[x] == -1) { nodes[cur].ch[x] = nodes.size(); nodes.emplace_back(); }
cur = nodes[cur].ch[x]; } nodes[cur].end = true; } bool search(string& s) { int cur =
0; for (char c : s) { int x = c - 'a'; if (nodes[cur].ch[x] == -1) return false; cur =
nodes[cur].ch[x]; } return nodes[cur].end; } };
```

11.3 — Hashing de structures

- **Unordered_map** avec custom hash pour éviter les anti-hash tests sur Codeforces.
- **Polynomial rolling hash** pour comparer des sous-chaînes en O(1).
- **Hashing de tuples** : hash(a,b,c) = a*P^2 + b*P + c mod M.

PARTIE 3 — TECHNIQUES FONDAMENTALES

Chapitre 12 — Two Pointers & Sliding Window

12.1 — Two Pointers classique

Deux pointeurs l et r sur un tableau trié. La clé : la propriété doit être MONOTONE (si [l,r] valide et on étend, toujours valide ou invalide dans un sens prévisible).

```
// Compter les paires (i,j) avec i<j et a[i]+a[j] == target (array trié) int count_pairs(vector<int>& a, int target) { int l = 0, r = a.size()-1, cnt = 0; while (l < r) { int s = a[l] + a[r]; if (s == target) { cnt++; l++; r--; } else if (s < target) l++; else r--; } return cnt; }
```

12.2 — Sliding Window (fenêtre glissante)

```
// Longueur du plus long sous-array sans doublon int longest_unique(vector<int>& a) { map<int,int> freq; int l = 0, ans = 0; for (int r = 0; r < a.size(); r++) { freq[a[r]]++; while (freq[a[r]] > 1) { // invariant violé freq[a[l]]--; if (freq[a[l]] == 0) freq.erase(a[l]); l++; } ans = max(ans, r - l + 1); } return ans; }
```

■ **CONSEIL : Compter les sous-arrays valides se terminant en r : souvent ans += r - l + 1.**

Chapitre 13 — Binary Search

13.1 — Binary Search sur la réponse

Technique fondamentale : si la réponse est une valeur dans [lo, hi] et qu'on peut vérifier en O(f(n)) si une réponse x est faisable, on trouve l'optimum en O(f(n) log(hi-lo)).

```
// Template general binary search // Cherche le plus petit x dans [lo,hi] tel que check(x) == true auto binary_search_answer = [&](ll lo, ll hi) -> ll { while (lo < hi) { ll mid = lo + (hi - lo) / 2; if (check(mid)) hi = mid; else lo = mid + 1; } return lo; }; // Cherche le plus grand x dans [lo,hi] tel que check(x) == true auto binary_search_last = [&](ll lo, ll hi) -> ll { while (lo < hi) { ll mid = lo + (hi - lo + 1) / 2; if (check(mid)) lo = mid; else hi = mid - 1; } return lo; };
```

■ **CONSEIL : Toujours réfléchir : cherche-t-on le PREMIER vrai ou le DERNIER vrai ? Ça change le template.**

13.2 — lower_bound et upper_bound en C++

```
vector<int> a = {1, 3, 3, 5, 7}; // lower_bound : premier élément >= val auto it = lower_bound(all(a), 3); // pointe vers a[1] // upper_bound : premier élément > val it = upper_bound(all(a), 3); // pointe vers a[3] // Nombre d'occurrences de val int cnt = upper_bound(all(a),3) - lower_bound(all(a),3); // = 2
```

Chapitre 14 — Greedy

14.1 — Qu'est-ce qu'un algorithme Greedy ?

Un algorithme greedy fait toujours le choix localement optimal à chaque étape, en espérant obtenir un optimum global. Contrairement au DP, il ne reconsidère jamais ses choix.

■■ ATTENTION : Sans PREUVE que le greedy est correct, il donne potentiellement un WA. Toujours prouver !

14.2 — Techniques de preuve

■ **Exchange argument** : supposons que la solution optimale diffère de la solution greedy à un moment. Montrer qu'échanger ces choix améliore ou maintient la qualité → contradiction.

■ **Preuve par induction** : montrer que le choix greedy à chaque étape maintient un invariant.

■ **Greedy stays ahead** : montrer que la solution greedy est au moins aussi bonne que toute autre à chaque étape.

14.3 — Patterns greedy classiques

Problème	Stratégie greedy	Complexité
Sélection d'activités (interval scheduling)	Trier par fin au plus tôt	O(n log n)
Remplissage de sac (fractionnaire)	Ratio valeur/poids décroissant	O(n log n)
Huffman coding	Priority queue, fusionner les 2 plus petits	O(n log n)
Problème de monnaie (certains systèmes)	Prendre la plus grande pièce possible	O(n)
Minimiser les retards (scheduling)	Trier par deadline	O(n log n)

PARTIE 4 — DYNAMIC PROGRAMMING (DP)

"La DP, c'est l'art de ne jamais résoudre le même sous-problème deux fois."

Chapitre 16 — Introduction & Cadre Général

16.1 — Les 4 étapes pour résoudre un problème DP

- **Étape 1 — Définir l'état** : dp[i] signifie quoi exactement ? C'est l'étape la plus importante.
- **Étape 2 — Écrire la transition** : comment dp[i] se calcule à partir des états précédents ?
- **Étape 3 — Définir le cas de base** : dp[0] = ?, dp[1] = ?
- **Étape 4 — Calculer la réponse** : c'est quoi l'état final voulu ?

16.2 — Top-down vs Bottom-up

	Top-down (Mémoïsation)	Bottom-up (Tabulation)
Code	Récursif + memo	Itératif + tableau
Avantage	Naturel, calcule seulement ce qu'il faut	Plus rapide, pas de stack overflow
Inconvénient	Overhead récursion, stack overflow	Doit calculer tous les états
Quand	États complexes/sparse	États linéaires/denses

Chapitre 17 — DP Classiques

17.1 — Longest Increasing Subsequence (LIS)

dp[i] = longueur de la LIS se terminant en a[i].

O(n^2) naïf, O(n log n) avec binary search :

```
// LIS en O(n log n) int LIS(vector<int>& a) { vector<int> dp; // dp[i] = plus petit element terminal d'une IS de longueur i+1 for (int x : a) { auto it = lower_bound(all(dp), x); if (it == dp.end()) dp.push_back(x); else *it = x; } return dp.size(); }
```

17.2 — Knapsack (Sac à dos)

```
// 0-1 Knapsack : n objets (poids w[i], valeur v[i]), capacité W // dp[j] = valeur max avec capacité j vector<ll> dp(W+1, 0); for (int i = 0; i < n; i++) for (int j = W; j >= w[i]; j--) // IMPORTANT : sens inverse pour 0-1 dp[j] = max(dp[j], dp[j-w[i]] + v[i]); // Unbounded Knapsack (objets illimités) for (int i = 0; i < n; i++) for (int j = w[i]; j <= W; j++) // sens normal dp[j] = max(dp[j], dp[j-w[i]] + v[i]);
```

17.3 — Longest Common Subsequence (LCS)

```
// LCS de s et t : O(nm) int LCS(string& s, string& t) { int n = s.size(), m = t.size(); vector<vector<int>> dp(n+1, vector<int>(m+1, 0)); for (int i = 1; i <= n; i++) for (int j = 1; j <= m; j++) dp[i][j] = (s[i-1]==t[j-1]) ? dp[i-1][j-1]+1 : max(dp[i-1][j], dp[i][j-1]); return dp[n][m]; }
```

Chapitre 18 — DP sur Intervalles & Arbres

18.1 — DP sur intervalles

dp[l][r] = réponse optimale pour le sous-problème sur l'intervalle [l,r].

Ordre de calcul : par longueur d'intervalle croissante (len = r - l + 1).

```
// Matrix Chain Multiplication // dp[l][r] = cout min pour multiplier matrices l..r for (int len = 2; len <= n; len++) { for (int l = 0; l+len-1 < n; l++) { int r = l + len - 1; dp[l][r] = LLONG_MAX; for (int k = l; k < r; k++) { ll cost = dp[l][k] + dp[k+1][r] + (ll)p[l]*p[k+1]*p[r+1]; dp[l][r] = min(dp[l][r], cost); } } }
```

18.2 — Tree DP

DP sur un arbre enraciné. On calcule dp[v] en utilisant les dp des fils de v. DFS post-order.

```
// Exemple : taille du plus grand sous-arbre independant vector<int> dp(n, 1); function<void(int,int)> dfs = [&](int v, int par) { for (int u : adj[v]) { if (u == par) continue; dfs(u, v); dp[v] += dp[u]; // dp[v] = taille du sous-arbre de v } };
```

Chapitre 19 — DP avec Bitmask

Utilisé quand n ≤ 20 et qu'on veut représenter des sous-ensembles. 2^20 = ~10^6 états.

```
// Travelling Salesman Problem (TSP) // dp[mask][v] = cout min pour visiter les villes dans mask, se terminant en v int n; // nombre de villes vector<vector<int>> dist(n, vector<int>(n, INF)); // distances vector<vector<int>> dp(1<<n, vector<int>(n, INF)); dp[1][0] = 0; // depart en ville 0 for (int mask = 1; mask < (1<<n); mask++) { for (int v = 0; v < n; v++) { if (!(mask >> v & 1)) continue; for (int u = 0; u < n; u++) { if (mask >> u & 1) continue; int nmask = mask | (1 << u); dp[nmask][u] = min(dp[nmask][u], dp[mask][v] + dist[v][u]); } } } // Réponse : min sur v de dp[(1<<n)-1][v] + dist[v][0]
```

Chapitre 20 — Optimisations DP

20.1 — Convex Hull Trick (CHT)

Optimise des DP de forme : dp[i] = min(dp[j] + a[j]*b[i] + c[i]) de O(n^2) à O(n log n) ou O(n).

- ■ Applicable quand la transition ressemble à évaluer des fonctions linéaires.
- ■ Li Chao Tree : généralisation, O(n log n) sans condition de monotonie.
- ■ SMAWK algorithm : O(n) pour matrices totalement monotones.

20.2 — Divide and Conquer DP

Quand opt[i][j] ≤ opt[i][j+1] (condition de monotonie), on peut optimiser certaines DP O(n^3) en O(n^2 log n) ou O(n^2).

20.3 — Optimisation par file monotone

dp[i] = min/max sur une fenêtre glissante de j → utiliser une deque monotone pour O(n).

PARTIE 5 — GRAPHES

"Tout problème complexe peut être modélisé comme un graphe."

Chapitre 21 — Représentation & Parcours

21.1 — Représentation des graphes

Représentation	Espace	Itérer voisins	Quand utiliser
Liste d'adjacence	O(V+E)	O(deg(v))	Graphes creux (standard)
Matrice d'adjacence	O(V^2)	O(V)	Graphes denses, V ≤ 1000
Liste d'arêtes	O(E)	O(E)	Kruskal, algo sur arêtes

21.2 — BFS (Breadth-First Search)

Parcours en largeur. Trouve le plus court chemin en nombre d'arêtes (graphe non pondéré) en O(V+E).

```
void bfs(int start, vector<vector<int>>& adj, int n) { vector<int> dist(n, -1);
queue<int> q; dist[start] = 0; q.push(start); while (!q.empty()) { int v = q.front();
q.pop(); for (int u : adj[v]) { if (dist[u] == -1) { dist[u] = dist[v] + 1; q.push(u);
} } } }
```

21.3 — DFS et ses applications

- **Détection de cycles** : dans un graphe non orienté, si on visite un voisin déjà visité (hors parent).
- **Composantes connexes** : lancer DFS depuis chaque nœud non visité.
- **Tri topologique** : ordre de finition inversé du DFS.
- **Euler tour** : numéroter les nœuds à l'entrée/sortie du DFS pour les requêtes sur sous-arbres.

Chapitre 22 — Plus Courts Chemins

22.1 — Dijkstra

SSSP (Single Source Shortest Path) pour graphes à poids positifs. O((V+E) log V).

```
void dijkstra(int src, vector<vector<pair<int,int>>>& adj, vector<ll>& dist) { int n
= adj.size(); dist.assign(n, LLONG_MAX); priority_queue<pair<ll,int>,
vector<pair<ll,int>>, greater<>> pq; dist[src] = 0; pq.push({0, src}); while
(!pq.empty()) { auto [d, v] = pq.top(); pq.pop(); if (d > dist[v]) continue; // noeud
deja traite for (auto [w, u] : adj[v]) { if (dist[v] + w < dist[u]) { dist[u] =
dist[v] + w; pq.push({dist[u], u}); } } } }
```

■■ ATTENTION : Dijkstra NE fonctionne PAS avec des poids négatifs → utiliser Bellman-Ford.

22.2 — Bellman-Ford

SSSP avec poids négatifs. O(VE). Détecte aussi les cycles négatifs.

```
bool bellman_ford(int src, int n, vector<tuple<int,int,int>>& edges, vector<ll>& dist) { dist.assign(n, LLONG_MAX); dist[src] = 0; for (int i = 0; i < n-1; i++) for (auto [u,v,w] : edges) if (dist[u] != LLONG_MAX && dist[u]+w < dist[v]) dist[v] = dist[u] + w; // Vérifier cycles négatifs for (auto [u,v,w] : edges) if (dist[u] != LLONG_MAX && dist[u]+w < dist[v]) return false; // cycle negatif return true; }
```

22.3 — Floyd-Warshall (All-Pairs)

APSP (All Pairs Shortest Path) en O(V^3). Pour V ≤ 500.

```
// dist[i][j] initialisé à INF, dist[i][i] = 0, dist[u][v] = poids(u,v) for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) if (dist[i][k] < INF && dist[k][j] < INF) dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
```

Chapitre 23 — Arbres Couvrants (MST)

23.1 — Algorithme de Kruskal

Trier les arêtes par poids croissant, ajouter si elles ne créent pas de cycle (DSU). O(E log E).

```
ll kruskal(int n, vector<tuple<int,int,int>>& edges) { sort(all(edges)); DSU dsu(n); ll total = 0; for (auto [w, u, v] : edges) if (dsu.unite(u, v)) total += w; return total; }
```

Chapitre 24 — Graphes Avancés

24.1 — Composantes Fortement Connexes (SCC)

Kosaraju : 2 DFS. Tarjan : 1 DFS avec stack. Utilisé pour 2-SAT, condensation de graphe.

24.2 — Bridges & Articulation Points

Bridge : arête dont la suppression déconnecte le graphe. Articulation point : sommet dont la suppression déconnecte.

Algorithme de Tarjan : O(V+E) avec disc[] et low[].

24.3 — 2-SAT

Résoudre des formules booléennes en forme (a ∨ b) en O(n+m) via SCC. Très fréquent en Div. 1 D/E.

Chapitre 25 — Flots & Matching

25.1 — Max-Flow (Flot Maximum)

Ford-Fulkerson O(Ef) → Edmonds-Karp O(VE^2) → Dinic O(V^2 E) (recommandé).

Théorème Max-Flow Min-Cut : le flot maximum = capacité du coupe minimum.

25.2 — Matching biparti

Hopcroft-Karp : O(E√V). Application : assignation tâches-travailleurs, couverture minimale.

PARTIE 6 — ALGORITHMES SUR LES CHAÎNES

Chapitre 26 — Hashing de Chaînes

Comparer des sous-chaînes en O(1) après O(n) de prétraitement. Deux hashes pour réduire les collisions.

```
struct StringHash { vector<ll> h, pw; ll MOD = 1e9+7, BASE = 131; StringHash(string& s) : h(s.size()+1, 0), pw(s.size()+1, 1) { for (int i = 0; i < s.size(); i++) { h[i+1] = (h[i] * BASE + s[i]) % MOD; pw[i+1] = pw[i] * BASE % MOD; } } // Hash de s[l..r] ll get(int l, int r) { return (h[r+1] - h[l] * pw[r-l+1] % MOD + MOD * 2) % MOD; } };
```

Chapitre 27 — KMP & Z-function

27.1 — Z-function

z[i] = longueur du plus long préfixe de s qui commence en s[i]. O(n).

```
vector<int> z_function(string& s) { int n = s.size(); vector<int> z(n, 0); z[0] = n; int l = 0, r = 0; for (int i = 1; i < n; i++) { if (i < r) z[i] = min(r-i, z[i-l]); while (i+z[i] < n && s[z[i]] == s[i+z[i]]) z[i]++; if (i+z[i] > r) { l = i; r = i+z[i]; } } return z; } // Chercher pattern p dans texte t : z_function(p + "#" + t)
```

27.2 — KMP (Knuth-Morris-Pratt)

Failure function π[i] = longueur du plus long préfixe propre de s[0..i] qui est aussi suffixe. O(n+m) pour la recherche.

Chapitre 28 — Suffix Array

Tableau de tous les suffixes triés lexicographiquement. Construction en O(n log n) ou O(n) avec SA-IS.

Avec LCP array : permet de résoudre des problèmes complexes sur les sous-chaînes en O(n log n).

- ☐ Nombre de sous-chaînes distinctes = n(n+1)/2 - Σlcp[i]
- ☐ Plus longue sous-chaîne commune à k chaînes : binary search + sliding window sur LCP
- ☐ Nombre d'occurrences d'un pattern : binary search sur le suffix array

Chapitre 29 — Aho-Corasick

Extension du Trie avec des failure links (comme KMP). Recherche de k patterns simultanément en O(n + Σ|patterns| + occurrences).

Applications : filtre de contenu, bioinformatique, détection de plagiat.

PARTIE 7 — GÉOMÉTRIE COMPUTATIONNELLE

Chapitre 30 — Primitives Géométriques

30.1 — Produit vectoriel (Cross Product)

Fondement de la géométrie 2D en CP. cross(A,B,C) > 0 : C est à gauche de AB. = 0 : colinéaires. < 0 : à droite.

```
typedef long long ll; struct Point { ll x, y; }; ll cross(Point O, Point A, Point B) { return (A.x-O.x)*(B.y-O.y) - (A.y-O.y)*(B.x-O.x); } // Aire du triangle OAB (en *2 pour eviter les flottants) ll area2 = abs(cross(O, A, B)); // Est-ce que le point P est à gauche de la ligne AB ? bool left_of(Point A, Point B, Point P) { return cross(A, B, P) > 0; }
```

30.2 — Intersection de segments

Deux segments AB et CD s'intersectent si et seulement si A et B sont de côtés opposés de CD, ET C et D de côtés opposés de AB.

Chapitre 31 — Convex Hull

Enveloppe convexe : le plus petit polygone convexe contenant tous les points. Andrew's Monotone Chain : O(n log n).

```
vector<Point> convex_hull(vector<Point> pts) { int n = pts.size(); sort(all(pts), [](Point a, Point b){ return a.x < b.x || (a.x == b.x && a.y < b.y); }); vector<Point> hull; // Lower hull for (int i = 0; i < n; i++) { while (hull.size() >= 2 && cross(hull[hull.size()-2], hull.back(), pts[i]) <= 0) hull.pop_back(); hull.push_back(pts[i]); } // Upper hull int lower_size = hull.size(); for (int i = n-2; i >= 0; i--) { while (hull.size() > lower_size && cross(hull[hull.size()-2], hull.back(), pts[i]) <= 0) hull.pop_back(); hull.push_back(pts[i]); } hull.pop_back(); return hull; }
```

Chapitre 32 — Distances & Plus Proche Paire

Plus proche paire de points en O(n log n) : Divide & Conquer ou approche avec tri.

PARTIE 8 — TECHNIQUES AVANCÉES

Chapitre 33 — Small-to-Large & DSU on Tree

33.1 — Small-to-Large Merging

Lors de la fusion de deux sets/maps, toujours itérer sur le plus petit et insérer dans le plus grand. Complexité totale : O(n log^2 n).

```
// Fusion small-to-large : chaque element est bougé O(log n) fois au total // car la taille double à chaque fusion dont il fait partie void merge_sets(set<int>& big, set<int>& small) { if (big.size() < small.size()) swap(big, small); for (int x : small) big.insert(x); small.clear(); }
```

33.2 — DSU on Tree (Euler Tour + Segment Tree)

Pour répondre à des requêtes sur des sous-arbres : numéroter les nœuds avec l'Euler tour pour transformer le sous-arbre de v en un intervalle [in[v], out[v]] dans le tableau.

```
int timer = 0; vector<int> in_time(n), out_time(n); function<void(int,int)> dfs = [&](int v, int par) { in_time[v] = timer++; for (int u : adj[v]) if (u != par) dfs(u, v); out_time[v] = timer - 1; }; // Maintenant: le sous-arbre de v = intervalle [in_time[v], out_time[v]] // Requête: segment tree sur cet intervalle !
```

Chapitre 34 — Mo's Algorithm

Algorithme offline pour des requêtes sur des intervalles [l,r]. Trier les requêtes avec l'ordre de Mo (blocs de √n) pour que les pointeurs l et r bougent au total O((n+q)√n) fois.

```
// Ordre de Mo : trier par (l/block, r) avec block = sqrt(n) int block = sqrt(n); sort(queries.begin(), queries.end(), [&](auto& a, auto& b) { int ba = a.l / block, bb = b.l / block; if (ba != bb) return ba < bb; return (ba & 1) ? (a.r > b.r) : (a.r < b.r); // optimisation Hilbert }); int cur_l = 0, cur_r = -1; for (auto& q : queries) { while (cur_r < q.r) add(++cur_r); while (cur_l > q.l) add(--cur_l); while (cur_r > q.r) remove(cur_r--); while (cur_l < q.l) remove(cur_l++); ans[q.id] = current_answer(); }
```

Chapitre 35 — Techniques de Randomisation

- **Random shuffle** avant d'appliquer un algorithme pour éviter les pires cas.
- **Hashing aléatoire** : choisir une base et un modulus aléatoires pour éviter les anti-hash.
- **Probabilistic primality** : Miller-Rabin pour tester les grands premiers.
- **Randomized algorithms** : QuickSort randomisé, algorithme de Karger (coupe minimale).

Chapitre 36 — Théorie des Nombres Avancée

36.1 — Théorème Chinois des Restes (CRT)

Trouver x tel que x ≡ r1 (mod m1) et x ≡ r2 (mod m2), si gcd(m1,m2)=1. Applications : calendriers, cryptographie, DP sur moduli.

36.2 — Baby-Step Giant-Step (BSGS)

Résoudre a^x ≡ b (mod p) en O(√p). Généralisation du logarithme discret.

36.3 — Racine primitive et groupe multiplicatif

Si p est premier, il existe une racine primitive g telle que g^1, g^2, ..., g^(p-1) sont tous distincts mod p. Utile pour la NTT (Number Theoretic Transform) et les convolutions.

PARTIE 9 — STRATÉGIE & COMPÉTITION

"Le code parfait ne sert à rien si tu choisis le mauvais algorithme."

Chapitre 37 — Lire et Analyser un Problème

37.1 — Les 5 étapes d'analyse

- **1. Lire les contraintes EN PREMIER** : n ≤ 10^5 → O(n log n) max. n ≤ 500 → O(n^2) OK.
- **2. Identifier le type de sortie** : oui/non, minimum, maximum, compter, construire.
- **3. Chercher la structure** : tableau trié ? Graphe ? Arbre ? Chaîne ?
- **4. Tester les cas simples** : n=1, n=2, tous identiques, tous distincts.
- **5. Trouver la propriété clé** : quelle observation mathématique simplifie tout ?

37.2 — Comment identifier le type d'algorithme

Signal dans l'énoncé	Type d'algorithme probable
'minimum/maximum' + contrainte relâchée	Greedy ou DP
Sous-tableaux, sous-chaînes	Two pointers, Sliding window, DP
Chemin dans un graphe	BFS/DFS/Dijkstra
Requêtes multiples sur intervalles	Segment Tree, BIT, Mo's
n ≤ 20	Bitmask DP, backtracking
Modulo dans la réponse	Combinatoire, DP modulaire
'Existe-t-il' ou 'est-il possible'	BFS, 2-SAT, greedy

Chapitre 38 — Plan d'Entraînement Annuel

38.1 — Roadmap par niveau

Phase	Durée	Objectif Rating	Focus principal
Débutant	0-3 mois	< 1200 → 1200	Bases C++, complexité, Div.2 A/B
Newbie→Pupil	3-6 mois	1200 → 1400	Greedy, Binary Search, Two Pointers
Pupil→Specialist	6-12 mois	1400 → 1600	DP classiques, Graphes BFS/DFS

Specialist→Expert	1-2 ans	1600 → 1900	Segment Tree, Dijkstra, DP avancée
Expert→CM	2-3 ans	1900 → 2100	Flots, Strings, Géométrie, DP opt.
CM→Master	3-4 ans	2100 → 2400	Techniques avancées, construction
Master→GM	4+ ans	2400+	Top-level, originalité, vitesse

38.2 — Plan hebdomadaire recommandé

- **Lundi-Vendredi** : 1-2 problèmes par jour dans ta zone de confort +1 niveau.
- **Samedi** : participer à un contest (Codeforces, AtCoder, etc.).
- **Dimanche** : upsolving du contest — résoudre les problèmes que tu n'as pas résolus.
- **Règle des 45 min** : si pas d'idée après 45 min → lire l'editorial → comprendre → recoder.
- **Carnet des erreurs** : noter chaque WA avec la raison. Patterns récurrents → travail ciblé.

38.3 — Ressources indispensables

Ressource	Type	Niveau
cp-algorithms.com	Référence algorithmes	Tous niveaux
CSES Problem Set (cses.fi)	300 problèmes classiques	Débutant → Expert
Codeforces (codeforces.com)	Contests + archive	Tous niveaux
AtCoder (atcoder.jp)	Contests haute qualité	Intermédiaire → Avancé
Competitive Programmer's Handbook	Livre gratuit (Laaksonen)	Débutant → Expert
Introduction to Algorithms (CLRS)	Livre théorique	Expert+
Codeforces EDU section	Cours structurés	Débutant → Expert
LC Hard + editorial study	LeetCode difficile	Intermédiaire

Chapitre 39 — Stratégie en Contest

39.1 — Codeforces Div. 2

- **0-5 min** : lire TOUS les énoncés rapidement.
- **5-20 min** : résoudre A en moins de 10 min si possible.
- **20-60 min** : B et C.
- **60-120 min** : D et upsolving si A/B/C terminés.

- **Toujours** : tester les edge cases avant de soumettre (n=1, overflow, limites).
- **Hacks** (Div. 1+2) : soumettre vite pour avoir le droit de hacker les autres.

39.2 — Gestion du stress et des WA

- WA on test 1 → faute de frappe ou mauvaise compréhension. Relire l'énoncé.
- WA on test 2/3 → edge cases. Tester n=1, n=2, cas tous identiques.
- TLE → vérifier la complexité, chercher une optimisation ou changer d'algorithme.
- MLE → reduce memory usage, éviter les vecteurs non nécessaires.
- Respirer : un WA ne détruit pas ton contest si tu corriges vite.

39.3 — ICPC vs IOI vs Codeforces

Compétition	Format	Équipe/Individuel	Style
Codeforces	2-3h, 5-7 problèmes	Individuel	Vitesse + diversité
IOI	2 jours, 3 problèmes/jour	Individuel	Profondeur + partiels
ICPC	5h, 12-13 problèmes	Équipe de 3	Coordination + breadth
Google Code Jam	Phases multiples	Individuel	Diversité algorithmes

---

Chapitre 40 — Vers le Niveau Grandmaster

40.1 — Ce qui sépare les GM des autres

- **Vitesse de pattern recognition** : voir le type de problème en 30 secondes.
- **Portfolio de techniques** : 50+ techniques maîtrisées, pas juste connues.
- **Capacité de construction** : créer des algorithmes sur mesure, pas juste appliquer.
- **Debug ultra-rapide** : trouver un bug en < 5 min via analyse, pas trial-and-error.
- **Intuition mathématique** : sentir qu'une propriété est vraie avant de la prouver.

40.2 — Méthode d'étude des GM

- Résoudre des problèmes Div. 1 D/E en upsolving, pas en contest.
- Étudier les solutions des top coders après chaque contest, même si tu as AC.
- Implémenter chaque technique au moins 3-5 fois jusqu'à la maîtriser.
- Créer ses propres problèmes : oblige à comprendre en profondeur.
- Lire les papiers académiques sur les algorithmes avancés.

40.3 — La vérité sur tourist (Gennady Korotkevich)

Le meilleur joueur de tous les temps a commencé à 6 ans et a participé à l'IOI à 11 ans. Ce n'est pas du génie pur — c'est 15 ans de pratique intense, avec une méthode rigoureuse et une passion authentique.

■ **CONSEIL : Tu n'as pas besoin d'être tourist. Être dans le top 1000 mondial est déjà un exploit extraordinaire.**

ANNEXE — RÉFÉRENCE RAPIDE

Complexités par catégorie — Cheat Sheet

Algorithme	Complexité Temps	Complexité Espace
BFS / DFS	O(V+E)	O(V)
Dijkstra (heap)	O((V+E) log V)	O(V+E)
Bellman-Ford	O(VE)	O(V)
Floyd-Warshall	O(V^3)	O(V^2)
Kruskal MST	O(E log E)	O(V+E)
Segment Tree	O(n) build, O(log n) update/query	O(n)
Binary Indexed Tree	O(n) build, O(log n) update/query	O(n)
Sparse Table	O(n log n) build, O(1) query	O(n log n)
Union-Find (DSU)	O(α(n)) ≈ O(1) par op.	O(n)
KMP / Z-function	O(n+m)	O(n)
Suffix Array	O(n log n)	O(n)
Aho-Corasick	O(Σ|patterns| + n + occ)	O(Σ|patterns|)
Convex Hull	O(n log n)	O(n)
LIS	O(n log n)	O(n)
LCS	O(nm)	O(nm)
0-1 Knapsack	O(nW)	O(W)
Bitmask DP	O(2^n · n)	O(2^n)
Mo's Algorithm	O((n+q) √n)	O(n+q)

Valeurs critiques à retenir

Valeur	Signification
10^9 + 7	Premier standard pour le modulo

10^9 + 9	Second premier pour double hashing
1 << 20 ≈ 10^6	Bitmask max en pratique
1 << 30 ≈ 10^9	INT_MAX ≈ 2.1 × 10^9
1LL << 60 ≈ 10^18	LLONG_MAX ≈ 9.2 × 10^18
√(10^18) ≈ 10^9	Factorisation jusqu'à √n
log2(10^9) ≈ 30	Profondeur d'un Segment Tree pour n=10^9
log2(2×10^5) ≈ 18	Profondeur pour n=2×10^5

Ce guide est un point de départ, pas une fin.

La vraie maîtrise vient de la pratique quotidienne, de la curiosité et de la persévérance.

Bonne chance dans ton voyage vers le sommet. ■